

7.3 Elementary Row Operations



Core skills you'll master in this section:

- ▶ Elementary row operations: the three basic operations – row swapping, row scaling, and row sum.
- ▶ The augmented matrix: writing a system of linear equations as $[A | b]$.
- ▶ Solving systems of linear equations: reducing the augmented matrix to a simplified form using elementary row operations.
- ▶ Finding the inverse matrix using elementary row operations.
- ▶ Implementing Gaussian elimination and inverse-matrix solving in Python.

This section introduces elementary row operations and their applications in solving systems of linear equations and finding matrix inverses. First, using the chicken-and-rabbit problem, we introduce the concept of the augmented matrix and progressively apply elementary row operations – row scaling, row swapping, and row sum – to reduce the system to a simplified form. Next, using the same row-operation methods, we reduce the matrix A to the identity matrix, thereby finding its inverse. Throughout this process, we observe how the equations, the matrix, the augmented matrix, and the linear combination all change, giving a visual and intuitive picture of the role elementary row operations play in mathematical computation.

Elementary Row Operations

The first section of this chapter mentioned elementary row operations. Elementary row operations are used to simplify a matrix through its rows, and are especially common when solving systems of linear equations, computing the rank of a matrix, and finding matrix inverses.

Elementary row operations are basic operations performed on the rows of a matrix, including:

- ▶ Row swapping: swapping two selected rows;
- ▶ Row scaling: multiplying all elements of a row by a nonzero scalar;
- ▶ Row sum: adding a nonzero multiple of one row to another.

Revisiting the Chicken-and-Rabbit Problem

Let's revisit a simplified version of the chicken-and-rabbit problem.

There are x_1 chickens and x_2 rabbits in the cage, with 3 heads and 8 feet in total; written as a system of linear equations,

$$\begin{cases} x_1 + x_2 = 3 \\ 2x_1 + 4x_2 = 8 \end{cases}$$

Figure 1 (a) shows the lines corresponding to the system of linear equations above; since the coefficients of the unknowns in both equations are nonzero, both lines are slanted.

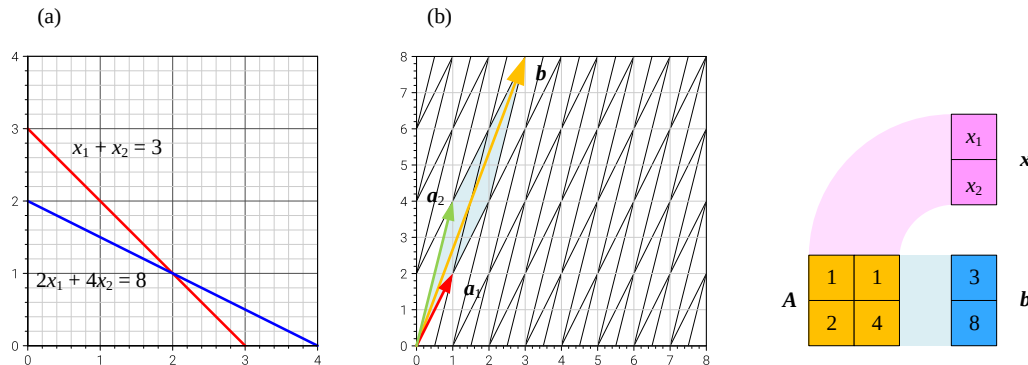


Figure 1. The Original System of Equations

written in matrix-multiplication form

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 2 & 4 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 3 \\ 8 \end{bmatrix}}_{\mathbf{b}}$$

Let's write this problem as an augmented matrix.

The augmented matrix is the new matrix formed by appending, to the right of the coefficient matrix $\mathbf{A}$, the constant vector $\mathbf{b}$, that is,

$$[\mathbf{A} \mid \mathbf{b}] = \left[\begin{array}{cc|c} 1 & 1 & 3 \\ 2 & 4 & 8 \end{array} \right]$$

⚠ Note that the vertical line between $\mathbf{A}$ and $\mathbf{b}$ in the expression above is sometimes omitted.

You'll soon see that applying elementary row operations to the augmented matrix can solve the system of linear equations $\mathbf{Ax} = \mathbf{b}$.

Based on the previous section, we can also write the coefficient matrix $\mathbf{A}$ as the column vectors $[\mathbf{a}_1, \mathbf{a}_2]$, so that the chicken-and-rabbit problem can be written in linear-combination form

$$x_1 \underbrace{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}_{\mathbf{a}_1} + x_2 \underbrace{\begin{bmatrix} 1 \\ 4 \end{bmatrix}}_{\mathbf{a}_2} = \underbrace{\begin{bmatrix} 3 \\ 8 \end{bmatrix}}_{\mathbf{b}}$$

Figure 1 (b) corresponds to the linear combination in the equation above.

Figure 1 (b), $\mathbf{a}_1$ and $\mathbf{a}_2$'s angle is less than 90° , but $\mathbf{a}_1$ and $\mathbf{a}_2$ are not collinear (linearly independent), so the resulting grid is a parallelogram. The determinant of the matrix $\mathbf{A}$ is $\det(\mathbf{A}) = 2$ (the area of a single parallelogram), so the matrix $\mathbf{A}$ is invertible, which also means the 2×2 matrix $\mathbf{A}$ has rank 2.

Next, let's use elementary row operations to solve this problem step by step, observing how the lines, the coefficient matrix, the augmented matrix, and the linear combination change throughout the process.

$$R_2/2 \rightarrow R_2$$

Applying row scaling $R_2/2 \rightarrow R_2$ to the second row, the system of linear equations becomes

$$\begin{cases} x_1 + x_2 = 3 \\ x_1 + 2x_2 = 4 \end{cases}$$

The row scaling $R_2/2 \rightarrow R_2$ means dividing every element in row 2 by 2 and replacing the original row 2 with the result, so that the pivot becomes 1, making later computations easier. A pivot (pivot element) is the leftmost nonzero coefficient in a row during the elimination process.

⚠ Note that a pivot is not the same as a main-diagonal element, since the pivot changes as row operations are applied.

Figure 2 (a) shows the two lines corresponding to the system of linear equations above; comparing Figure 1 (a) and Figure 2 (a), clearly row scaling does not change the position or shape of the lines.

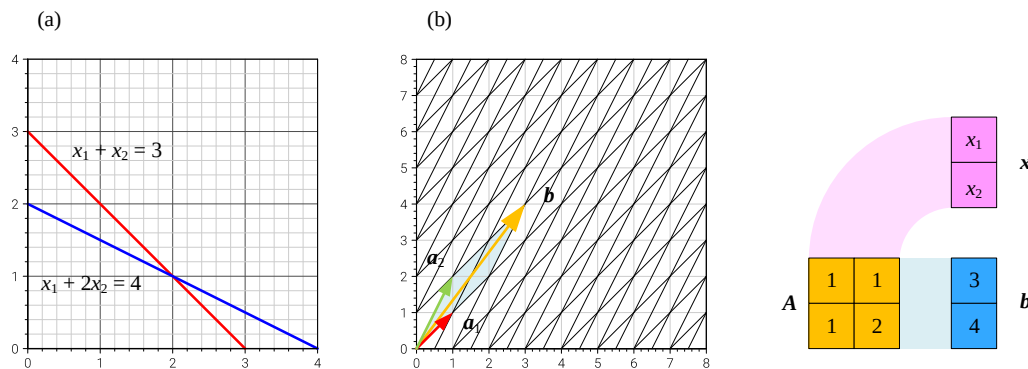


Figure 2. Row Scaling, $R_2/2 \rightarrow R_2$

the corresponding matrix multiplication becomes

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 3 \\ 4 \end{bmatrix}}_b$$

The augmented matrix becomes

$$[A \mid b] = \left[\begin{array}{cc|c} 1 & 1 & 3 \\ 1 & 2 & 4 \end{array} \right]$$

The linear combination becomes

$$x_1 \underbrace{\begin{bmatrix} 1 \\ 1 \end{bmatrix}}_{a_1} + x_2 \underbrace{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}_{a_2} = \underbrace{\begin{bmatrix} 3 \\ 4 \end{bmatrix}}_b$$

Figure 2 (b) shows the linear combination; row scaling has changed the matrix A 's column vectors, and the determinant becomes 1. The grid is still a parallelogram at this point.

$$R_2 - R_1 \rightarrow R_2$$

Applying row sum $R_2 - R_1 \rightarrow R_2$ – that is, subtracting row 1 from row 2 of and using the result to replace row 2 – eliminates x_1 from row 2. The system of linear equations becomes

$$\begin{cases} x_1 + x_2 = 3 \\ x_2 = 1 \end{cases}$$

As shown in Figure 3 (a), $x_2 = 1$ corresponds to a horizontal line, meaning the solution for x_2 has already appeared!

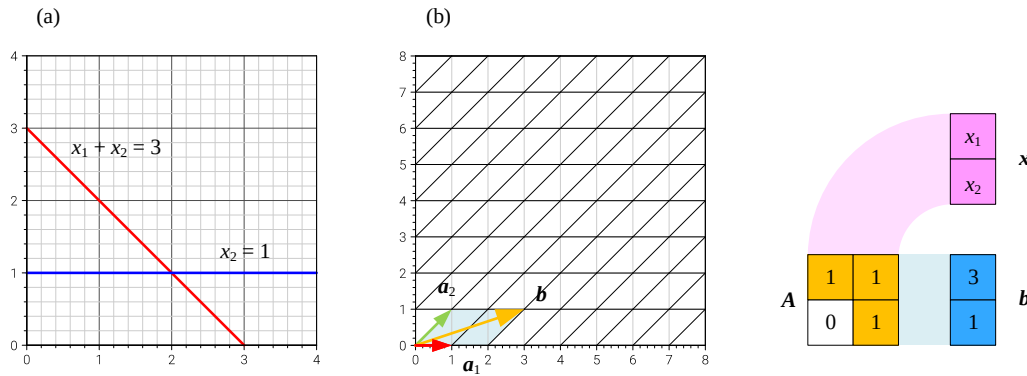


Figure 3. Row Sum, $R_2 - R_1 \rightarrow R_2$

the corresponding matrix multiplication becomes

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 3 \\ 1 \end{bmatrix}}_b$$

We find that the matrix A has already become an upper triangular matrix! The determinant of the matrix is the product of the main-diagonal elements, which is 1.

The augmented matrix becomes

$$[A | b] = \left[\begin{array}{cc|c} 1 & 1 & 3 \\ 0 & 1 & 1 \end{array} \right]$$

The linear combination becomes

$$x_1 \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{a_1} + x_2 \underbrace{\begin{bmatrix} 1 \\ 1 \end{bmatrix}}_{a_2} = \underbrace{\begin{bmatrix} 3 \\ 1 \end{bmatrix}}_b$$

As shown in Figure 3 (b), the parallelogram grid becomes a grid sheared along the horizontal axis.

$$R_1 - R_2 \rightarrow R_1$$

Applying row sum $R_1 - R_2 \rightarrow R_1$ – that is, subtracting the corresponding elements of row 2 from row 1 and using the result to replace the original row 1 – the system of linear equations becomes

$$\begin{cases} x_1 = 2 \\ x_2 = 1 \end{cases}$$

Looking at Figure 4 (a), the two lines have become vertical and horizontal, and the solutions for both x_1 and x_2 have appeared!

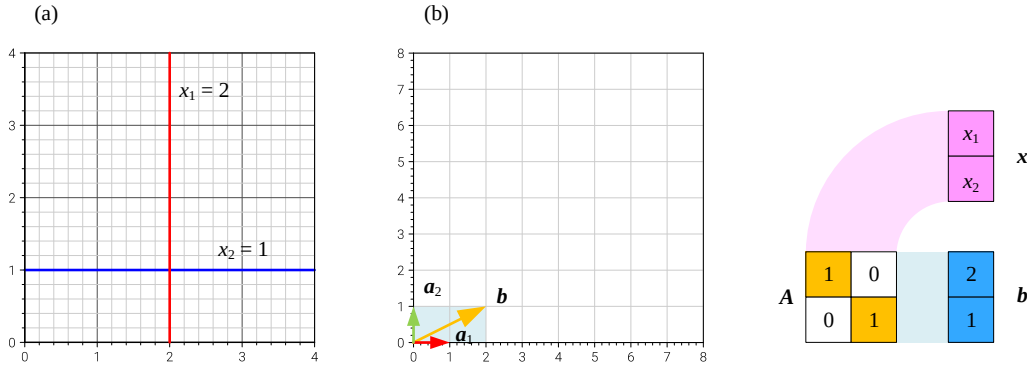


Figure 4. Row Sum, $R_1 - R_2 \rightarrow R_1$

The matrix multiplication becomes

$$\underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 2 \\ 1 \end{bmatrix}}_b$$

The matrix A has become the identity matrix. By the property of matrix multiplication with the identity matrix, the equation above can be simplified to

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 2 \\ 1 \end{bmatrix}}_b$$

The augmented matrix becomes

$$[A \mid b] = \left[\begin{array}{cc|c} 1 & 0 & 2 \\ 0 & 1 & 1 \end{array} \right]$$

The linear combination becomes

$$\underbrace{x_1}_{a_1} \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{a_1} + \underbrace{x_2}_{a_2} \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_{a_2} = \underbrace{\begin{bmatrix} 2 \\ 1 \end{bmatrix}}_b$$

As shown in Figure 4 (b), the grid has become a unit square; the two column vectors have become the standard basis $[e_1, e_2]$.

By solving the system of linear equations using elementary row operations, we observed the transformation of the equations, the matrix, the augmented matrix, and the linear combination (the parallelogram grid).

Programming: Solving Systems of Linear Equations with Elementary Row Operations

Code 1 is the Python code for solving a system of linear equations using elementary row operations. Let's go over the key statements below.

- a** Defines a function named `gauss_elimination` that takes two parameters: $\mathbf{A}$ is the coefficient matrix, a 2D NumPy array; $\mathbf{b}$ is the constant vector, a 1D NumPy array.
- b** Converts $\mathbf{A}$ and $\mathbf{b}$ to the float type, ensuring that integer arithmetic doesn't introduce errors during computation.
- c** Constructs the augmented matrix $[\mathbf{A}|\mathbf{b}]$; uses `b.reshape(-1, 1)` to convert $\mathbf{b}$ into a column vector, and then uses `np.hstack()` to horizontally stack $\mathbf{A}$ and $\mathbf{b}$ together, forming a new matrix of shape $n \times (n+1)$, $[\mathbf{A}|\mathbf{b}]$, stored in the variable `Ab`.
- d** Uses a for loop to convert $\mathbf{A}$ into an upper triangular matrix. The for loop iterates over the column index i of matrix $\mathbf{A}$ to control the elimination process.
- e** Finds the row (called the pivot row) containing the element with the largest absolute value, among the current column i and all rows below it. `np.abs(Ab[i:, i])` takes the absolute values of column i , `np.argmax` finds the index of the maximum among them, and adding i computes its true row number in `Ab`.
- f** Swaps the current row i with `max_row`, ensuring the diagonal element (the pivot) has as large an absolute value as possible, to reduce computational error.
- g** Uses row scaling to make the pivot coefficient 1.
- h** Uses a nested loop to eliminate the column- i elements of all rows below the current column i , setting them to 0.
- i** Uses row sum to eliminate the elements in column i . Specifically: `Ab[j, i]` is the coefficient of row j in the current column. `Ab[j, i] * Ab[i, i]` is the multiple needed to eliminate the element in row j , column i . `Ab[j, i] - Ab[j, i] * Ab[i, i]` is the new value after the row operation.
- j** Initializes the solution vector $\mathbf{x}$, an array of length n filled with zeros.
- k** Starting from the last row and working upward, solves for $x[i]$ row by row. `Ab[i, -1]` is the value of $\mathbf{b}$, that is, the last column of the augmented matrix. `np.dot(Ab[i, i+1:n], x[i+1:n])` computes the weighted sum of the already-known solutions, and `Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])` is the solution for the current row.
- 1** Defines $\mathbf{A}$ and $\mathbf{b}$, and calls the custom function to solve the system of linear equations.

Code 1. Solving a System of Linear Equations Using Elementary Row Operations |  LA_Ch07_03_01.ipynb

```

## 初始化
import numpy as np

## 自定义函数
a def gauss_elimination(A, b):
    # 确保使用浮点数计算
    b A = A.astype(float)
    b b = b.astype(float)
    b n = len(b)

    # 构造增广矩阵 [A|b]
    c Ab = np.hstack([A, b.reshape(-1, 1)])

    # 原始矩阵A > 上三角
    d for i in range(n):
        # 1) 行交换, 选择绝对值最大为主元
        # 主元: 主对角线元素
        e max_row = np.argmax(np.abs(Ab[i:, i])) + i
        f Ab[[i, max_row]] = Ab[[max_row, i]]
        # 交换行, 行 i <-> 行 max_row

        # 2) 行数乘, 归一化主元, 主元变为 1
        g Ab[i] = Ab[i] / Ab[i, i]

        # 3) 行倍加, 消元, 使主元下面所有行的 i列变为 0
        h for j in range(i+1, n):
            # Ab[j] -= Ab[j, i] * Ab[i]
            i Ab[j] = Ab[j] - Ab[j, i] * Ab[i]

    # 回代求解, 上三角矩阵 > 单位矩阵
    j x = np.zeros(n)
    k for i in range(n-1, -1, -1):
        x[i] = Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])

    return x

## 定义矩阵 A 和向量 b
l A = np.array([[1, 3, 6],
                [2, 7, 14],
                [0, 2, 5]])

b = np.array([25, 58, 19])

## 求解 Ax = b
x = gauss_elimination(A, b)

```

To help everyone understand this code, let's take the example given in Code 1 and compute it step by step following the logic above.

Constructing the Augmented Matrix

Based on the given A and b , we construct the augmented matrix

$$\begin{bmatrix} 1 & 3 & 6 & 25 \\ 2 & 7 & 14 & 58 \\ 0 & 2 & 5 & 19 \end{bmatrix}$$

This corresponds to the code .

First Column

Row swapping: looking at 's first column, the largest absolute value is in column 1, row 2; swap rows 1 and 2

$$\begin{bmatrix} 2 & 7 & 14 & 58 \\ 1 & 3 & 6 & 25 \\ 0 & 2 & 5 & 19 \end{bmatrix}$$


Row scaling: normalize 's pivot in column 1 (row 1, column 1)

$$\begin{bmatrix} 1 & 3.5 & 7 & 29 \\ 1 & 3 & 6 & 25 \\ 0 & 2 & 5 & 19 \end{bmatrix}$$

Row sum: eliminate the elements below the pivot in 's column 1, making them 0

$$\begin{bmatrix} 1 & 3.5 & 7 & 29 \\ 0 & -0.5 & -1 & -4 \\ 0 & 2 & 5 & 19 \end{bmatrix}$$

Second Column

Row swapping: looking at 's second column (the elements below the pivot), the largest absolute value is in column 2, row 3; swap rows 2 and 3

$$\begin{bmatrix} 1 & 3.5 & 7 & 29 \\ 0 & 2 & 5 & 19 \\ 0 & -0.5 & -1 & -4 \end{bmatrix}$$


Row scaling: normalize 's pivot in column 2 (row 2, column 2)

$$\begin{bmatrix} 1 & 3.5 & 7 & 29 \\ 0 & 1 & 2.5 & 9.5 \\ 0 & -0.5 & -1 & -4 \end{bmatrix}$$

Row sum: eliminate the elements below the pivot in 's column 2, making them 0

$$\begin{bmatrix} 1 & 3.5 & 7 & 29 \\ 0 & 1 & 2.5 & 9.5 \\ 0 & 0 & 0.25 & 0.75 \end{bmatrix}$$

Third Column

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

Since the matrix A has only 3 columns, this time we only need to apply row scaling: normalize 's pivot in column 3 (row 3, column 3)

$$\begin{bmatrix} 1 & 3.5 & 7 & 29 \\ 0 & 1 & 2.5 & 9.5 \\ 0 & 0 & 1 & 3 \end{bmatrix}$$

At this point, $\textcircled{d}$'s for loop has finished, and the matrix A has already been transformed into an upper triangular matrix.

Back Substitution

Back-substitution to solve is then especially simple

$$\begin{aligned} x_3 &= 3 \\ x_2 &= 9.5 - 2.5 \times 3 = 2 \\ x_1 &= 29 - 3.5 \times 2 - 7 \times 3 = 1 \end{aligned}$$

The computation above corresponds to $\textcircled{k}$. The process above can be summarized using Figure 5.

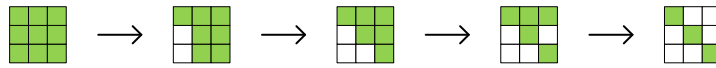


Figure 5. The Elementary Row Operations Process

Solving for the Inverse Matrix by Hand

Chapter 5 of this book introduced how to solve for a matrix inverse by hand using the adjugate-matrix method, and mentioned at the time that elementary row operations can also be used to solve for a matrix inverse by hand.

Next, let's use elementary row operations to find the inverse of the matrix in A .

First, we stack A together with the identity matrix I to form the augmented matrix

$$[A \mid I] = \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 2 & 4 & 0 & 1 \end{array} \right]$$

Our goal is to use elementary row operations to transform the square matrix A on the left side of the augmented matrix into I , so that the matrix on the right side is the inverse matrix A^{-1} ; that is, to use elementary row operations to transform the augmented matrix into

$$[I \mid A^{-1}]$$

Next, let's repeat the elementary row operations from before.

Applying row scaling $R_2/2 \rightarrow R_2$, the augmented matrix becomes

$$\left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1/2 \end{array} \right]$$

Applying row sum $R_2 - R_1 \rightarrow R_2$, the augmented matrix becomes

$$\left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & 1/2 \end{array} \right]$$

Finally, applying row sum $R_1 - R_2 \rightarrow R_1$, the augmented matrix becomes

$$\left[\begin{array}{cc|cc} 1 & 0 & 2 & -1/2 \\ 0 & 1 & -1 & 1/2 \end{array} \right]$$

At this point, the square matrix on the left side of the augmented matrix has become I , and the matrix on the right side is the inverse matrix A^{-1} .

Let's verify: matrix A times A^{-1}

$$\begin{bmatrix} 1 & 1 \\ 2 & 4 \end{bmatrix} @ \begin{bmatrix} 2 & -0.5 \\ -1 & 0.5 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

the inverse matrix A^{-1} times A

$$\begin{bmatrix} 2 & -0.5 \\ -1 & 0.5 \end{bmatrix} @ \begin{bmatrix} 1 & 1 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Programming: Solving for the Inverse Matrix with Elementary Row Operations

Code 2 solves for the inverse matrix using elementary row operations. Let's go over the key statements below.

a The line `if n != m:` checks whether A is a square matrix. A square matrix is defined as having an equal number of rows and columns, that is, $n == m$. If n is not equal to m , that means A is not a square matrix, so we print an error message and return `None`, indicating that this function cannot proceed.

b Uses `np.linalg.det(A)` to compute A 's determinant, and the line `if np.linalg.det(A) == 0:` checks whether it is 0. If the determinant equals 0, then A is a singular matrix with no inverse, so we print an error message and return `None`.

c Creates an identity matrix I of the same size as A . An identity matrix is a matrix that is all 1s on the main diagonal and all 0s everywhere else.

d The line `A_I = np.hstack((A, I))` horizontally stacks A and I together, forming an augmented matrix $[A | I]$. Mathematically, this matrix means that we perform the same row operations simultaneously on A and I , ultimately turning A into the identity matrix, at which point I becomes A 's inverse matrix.

e Enters the loop for `i in range(n):`, whose purpose is to apply Gauss-Jordan elimination to A_I , turning A into the identity matrix. The loop variable `i` represents the column index currently being processed.

f Finds the row `max_row` containing the element with the largest absolute value in the current column, then swaps row `i` and row `max_row` using `A_I[[i, max_row]] = A_I[[max_row, i]]`.

g Executes the line `A_I[i] = A_I[i] / A_I[i, i]`, whose purpose is to make the main-diagonal element of the current row 1. We divide all elements of row `i` by `A_I[i, i]`, so that the pivot becomes 1 and the other elements are scaled proportionally.

h Enters the nested loop for j in $\text{range}(n)$; whose purpose is to make all elements of column i in the other rows (besides the pivot) become 0. We use $i \neq j$ to ensure we don't modify the current pivot row, and then execute $A_I[j] -= A_I[j, i] * A_I[i]$, which eliminates row j using the current pivot row, making the element at row j , column i become 0.

i Uses $A_inv = A_I[:, n:]$ to extract the inverse of matrix A from the right-hand side.

Code 2. Solving for the Inverse Matrix Using Elementary Row Operations |  LA_Ch07_03_02.ipynb

```

## 初始化
import numpy as np

## 自定义函数
def inverse_matrix(A):

    # 确保 A 是 NumPy 数组并转换为浮点数类型
    A = np.array(A, dtype=float)

    # 获取矩阵的行数和列数
    n, m = A.shape

    # 判断是否为方阵
    if n != m:
        print("错误: 矩阵不是方阵, 无法求逆。")
        return None

    # 计算行列式, 判断是否可逆
    if np.linalg.det(A) == 0:
        print("错误: 矩阵的行列式为 0, 无法求逆。")
        return None

    # 创建单位矩阵 I
    I = np.eye(n)

    # 构造增广矩阵 [A | I]
    A_I = np.hstack((A, I))

    # 进行高斯-约当消元法
    for i in range(n):
        # 1) 行交换, 选取主元
        max_row = np.argmax(np.abs(A_I[i:, i])) + i
        A_I[[i, max_row]] = A_I[[max_row, i]]

        # 2) 行数乘, 归一化主元 (主对角线元素变为 1)
        A_I[i] = A_I[i] / A_I[i, i]

        # 3) 行加倍, 消元 (使主元列的其他元素变为 0)
        for j in range(n):
            if i != j:
                A_I[j] -= A_I[j, i] * A_I[i]

    # 提取逆矩阵
    A_inv = A_I[:, n:]

    return A_inv

## 定义矩阵 A
A = np.array([[1, 3, 6],
               [2, 7, 14],
               [0, 2, 5]])

## 计算逆矩阵
A_inv = inverse_matrix(A)

```





Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Use elementary row operations to solve the following systems of linear equations, and plot the lines and the linear-combination transformation throughout the process.

▶
$$\begin{cases} x_1 + x_2 = 4 \\ 2x_1 + 4x_2 = 12 \end{cases}$$

▶
$$\begin{cases} x_1 + 2x_2 + 3x_3 = 1 \\ 3x_1 + x_3 = 2 \\ x_1 + 2x_2 + x_3 = 3 \end{cases}$$

▶
$$\begin{cases} x_1 + x_2 = 2 \\ x_1 - x_3 = 1 \\ x_2 - x_3 = 0 \end{cases}$$

Q2. Use elementary row operations to find the inverse of the following matrices.

▶
$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

▶
$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix}$$

▶
$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & 1 & -1 \end{bmatrix}$$

▶
$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

Q3. Use elementary row operations to compute the rank of the matrices from the previous question.

Q4. What is an elementary matrix?

Q5. What is LU decomposition?

Q6. Study on your own how to perform LU decomposition using SciPy library functions.

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.lu.html>

Q7. What is LDL decomposition?

Q8. Study on your own how to perform LDL decomposition using SciPy library functions.

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.ldl.html>